



۸۱۰۱۰۳۰۹۸

پردازش اطلاعات کوانتومی
نام و نام خانوادگی: بهزاد جنتی



گزارش پروژه ۲

محل درج گزارش تمرین به همراه مراجع [۱]

بخش اول: الگوریتم Grover

الگوریتم Grover یکی از الگوریتم‌های مهم در محاسبات کوانتومی است که برای حل مسائل جستجوی بدون ساختار (مانند جستجوی یک عنصر خاص در یک لیست) با پیچیدگی زمانی بهینه طراحی شده است. این الگوریتم می‌تواند احتمال یافتن جواب درست را در فضای جستجو به صورت نمایی افزایش دهد.

مراحل اصلی الگوریتم Grover ۱. آماده‌سازی حالت اولیه (State Preparation): در ابتدا، یک حالت ابرپوشانی یکنواخت (Uniform Superposition) از تمام حالات ممکن ایجاد می‌شود. این کار با اعمال گیت‌های Hadamard روی تمام کیوبیت‌ها انجام می‌شود. - حالت اولیه به صورت زیر است:

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$$

که $N = 2^n$ تعداد حالات ممکن است.

۲. اوراکل (Oracle): - اوراکل یک مدار کوانتومی است که حالت درست (جواب) را مشخص می‌کند. این کار با تغییر فاز حالت موردنظر انجام می‌شود:

$$U_{\omega}|x\rangle = \begin{cases} -|x\rangle & \text{اگر } x = \omega, \\ |x\rangle & \text{در غیر این صورت.} \end{cases}$$

اوراکل به صورت یک ماتریس قطری عمل می‌کند که تنها برای حالت جواب یک فاز منفی اعمال می‌کند.

۳. اپراتور انتشار (Diffusion Operator): - این اپراتور که گاهی به آن "انعکاس درباره میانگین" (Reflection about the Mean) نیز گفته می‌شود، احتمال حالت جواب را تقویت کرده و احتمال سایر حالات را کاهش می‌دهد. - فرمول اپراتور انتشار:

$$U_s = 2|s\rangle\langle s| - I$$

۴. تکرار مراحل (Iterations): - اوراکل و انتشار به صورت متناوب و به تعداد مشخصی تکرار می‌شوند (t) تا احتمال یافتن جواب به حداکثر برسد. تعداد بهینه تکرارها به صورت زیر محاسبه می‌شود:

$$t = \left\lfloor \frac{\pi}{4} \sqrt{\frac{N}{m}} \right\rfloor$$

که N تعداد حالات و m تعداد جواب‌های ممکن است.

۵. اندازه‌گیری (Measurement): - در نهایت، حالت کوانتومی اندازه‌گیری می‌شود و حالت با بیشترین احتمال به عنوان جواب انتخاب می‌شود.

□

بخش دوم: حل سودوکو با استفاده از الگوریتم Grover در این بخش، الگوریتم Grover برای حل مسئله سودوکو به کار گرفته شده است. سودوکو یک مسئله جستجوی محدودیت‌دار است که در آن باید اعداد را در یک جدول قرار دهیم به طوری که قوانین خاصی رعایت شوند (مانند عدم تکرار اعداد در هر ردیف، ستون یا زیرشبکه).

مراحل حل سودوکو ۱. تعریف فضای جستجو:

- فضای جستجو شامل تمام حالات ممکن برای پر کردن جدول سودوکو است. - برای یک جدول $n \times n$ ، فضای جستجو برابر با $N = n^{n^2}$ است. با این حال، با توجه به مقادیر اولیه داده شده در جدول، این فضا کاهش می‌یابد. ۲. ایجاد اوراکل:

- اوراکل برای سودوکو به گونه‌ای طراحی شده است که تنها حالاتی را که تمام قوانین سودوکو را رعایت می‌کنند، "علامت‌گذاری" کند.

- قوانین سودوکو شامل موارد زیر است:

- هر عدد باید یک بار در هر ردیف ظاهر شود.

- هر عدد باید یک بار در هر ستون ظاهر شود.

- هر عدد باید یک بار در هر زیرشبکه ظاهر شود.

- اوراکل این قوانین را بررسی کرده و فاز حالت‌هایی که این قوانین را رعایت می‌کنند، منفی می‌کند.

۳. ایجاد اپراتور انتشار (Diffusion Operator): - این اپراتور مشابه حالت عمومی الگوریتم Grover است و احتمال حالت‌های علامت‌گذاری شده را تقویت می‌کند.

۴. اجرای الگوریتم Grover: - مراحل اوراکل و انتشار به صورت متناوب اجرا می‌شوند تا حالت جواب با احتمال بالا تقویت شود.

۵. اندازه‌گیری و یافتن جواب: - در نهایت، حالت کوانتومی اندازه‌گیری شده و جدول سودوکو کامل به دست می‌آید. جزئیات پیاده‌سازی در Qiskit در فایل ارائه شده، مراحل زیر برای حل سودوکو با استفاده از Qiskit انجام شده است: ایجاد اوراکل سودوکو

۱. بررسی قوانین سودوکو: - اوراکل به صورت مجموعه‌ای از گیت‌های منطقی طراحی شده است که قوانین سودوکو را بررسی می‌کنند.

- برای مثال، از گیت‌های XOR برای بررسی عدم تکرار اعداد استفاده می‌شود.

۲. مدار اوراکل: - اوراکل به صورت یک مدار کوانتومی طراحی شده که حالت‌های معتبر را علامت‌گذاری می‌کند.

ایجاد اپراتور انتشار

۱. حالت اولیه: - با استفاده از گیت‌های Hadamard، حالت اولیه ابرپوشانی یکنواخت ایجاد می‌شود.
 ۲. انعکاس درباره میانگین: - اپراتور انتشار با استفاده از گیت‌های Hadamard و یک گیت کنترل‌شده ایجاد می‌شود.
- اجرای الگوریتم

۱. مدار کامل: - مدار شامل اوراکل و اپراتور انتشار به تعداد مشخصی تکرار می‌شود.
 ۲. شبیه‌سازی: - مدار روی یک شبیه‌ساز کوانتومی اجرا می‌شود.
 ۳. اندازه‌گیری: - حالت نهایی اندازه‌گیری شده و جواب به دست می‌آید.
- نتیجه‌گیری حل سودوکو با استفاده از الگوریتم Grover نشان‌دهنده قدرت محاسبات کوانتومی در حل مسائل جستجوی پیچیده است. این روش با کاهش فضای جستجو و افزایش احتمال یافتن جواب، کارایی بالایی دارد. با این حال، محدودیت‌های سخت‌افزاری فعلی (مانند تعداد کیوبیت‌ها) ممکن است اجرای این الگوریتم را برای جدول‌های بزرگ‌تر دشوار کند.
- توضیحات داده شده بالا کلیاتی از حل مسئله سودوکو به کمک الگوریتم Grover را بیان می‌کرد حال در این قسمت به توضیح دقیق‌تر این پروژه می‌پردازیم.
- بخش یک توضیح مسئله:

یک سودوکو 2×2 داریم که می‌خواهیم به کمک الگوریتم Grover آن را حل کنیم.

- No column may contain the same value twice
- No row may contain the same value twice

If we assign each square in our sudoku to a variable like so:

V_0	V_1
V_2	V_3

we want our circuit to output a solution to this sudoku.

شکل ۱: 2×2 binary sudoku

به سادگی میتوان مانند شکل ۱ جدول را به چهار بیت (یا کیوبیت) تقسیم کرد و فضای مسئله را بر اساس این کیوبیت ها بررسی کرد. مشخص است که اگر v_0, v_1, v_2, v_3 هر کدام یک کیوبیت باشند، هر گونه مقداردهی به این متغیرها

یک روش برای پر کردن جدول است. حالت هایی از پر کردن جدول برای ما مطلوب است که خانه های درونی یک سطر یا ستون دارای مقدار متفاوتی باشند
همان طور که در شکل بالا و توضیحات مشخص است هیچ مقدار یکسانی در یک سطر یا ستون نباید داشته باشیم لذا به روابط زیر میرسیم:

```
v0 ≠ v1    # check along top row
v2 ≠ v3    # check along bottom row
v0 ≠ v2    # check down left column
v1 ≠ v3    # check down right column
```

شکل ۲: comparisons in a sudoku 2*2.

در این قسمت برای راحتی، مقایسات بالا رو در قالب یک لیست از کلازها به شکلی که در زیر مشاهده میکنید مینویسیم.

```
clause_list = [[0,1],
               [0,2],
               [1,3],
               [2,3]]
```

شکل ۳: نحوه تعریف clause list در qiskit

حال برای چک کردن کلازها نیاز داریم تا از گیت XOR همان طور که در بخش راهنمایی پروژه نیز آمده بود استفاده کنیم پس نیاز به تعریف آن در qiskit داریم.

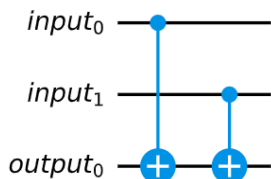
```
def XOR(qc, a, b, output):
    qc.cx(a, output)
    qc.cx(b, output)
```

شکل ۴: نحوه تعریف XOR در qiskit

حال جهت بررسی کلازهای مختلف باید گیت XOR زیر را برای تک تک مقایسات ذکر شده در کلاز لیست تکرار کنیم.

در شکل پایین تنها زمانی مقدار output0 به اصطلاح flip میشود که مقدار input2 ≠ input1 باشد.
حال در این مرحله گیت بالا رو برای تمام مقایسات گفته شده در بخش قبلی پیاده سازی میکنیم.

```
# We will use separate registers to name the bits
in_qubits = QuantumRegister(2, name='input')
out_qubit = QuantumRegister(1, name='output')
qc = QuantumCircuit(in_qubits, out_qubit)
XOR(qc, in_qubits[0], in_qubits[1], out_qubit)
qc.draw()
```



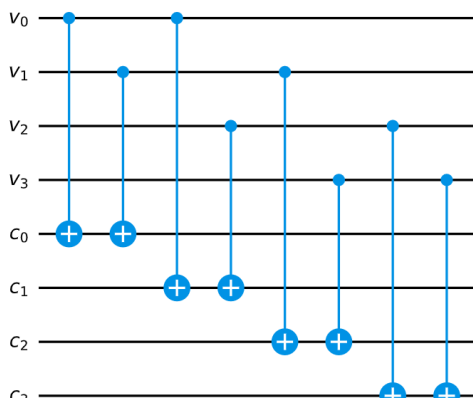
شکل ۵: استفاده از XOR برای مقایسه

```
# Create separate registers to name bits
var_qubits = QuantumRegister(4, name='v') # variable bits
clause_qubits = QuantumRegister(4, name='c') # bits to store clause-checks

# Create quantum circuit
qc = QuantumCircuit(var_qubits, clause_qubits)

# Use XOR gate to check each clause
i = 0
for clause in clause_list:
    XOR(qc, clause[0], clause[1], clause_qubits[i])
    i += 1
qc.draw()
```

23]:

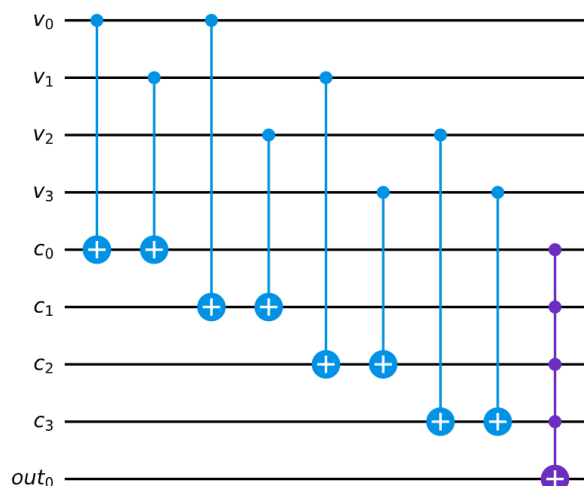


شکل ۶: استفاده از XOR برای تمام مقایسات

برای مرحله پایانی نیز برای اینکه خروجی بیت های c_0, c_1, c_2, c_3 همگی یک باشند باید تمام روابطی و شرایطی که تعریف کرده بودیم (روی v_0, v_1, v_2, v_3) برقرار باشند، به زبان دیگر باید جواب سودوکو را پیدا کرده باشیم (کلازهای تعریف شده satisfy شوند).

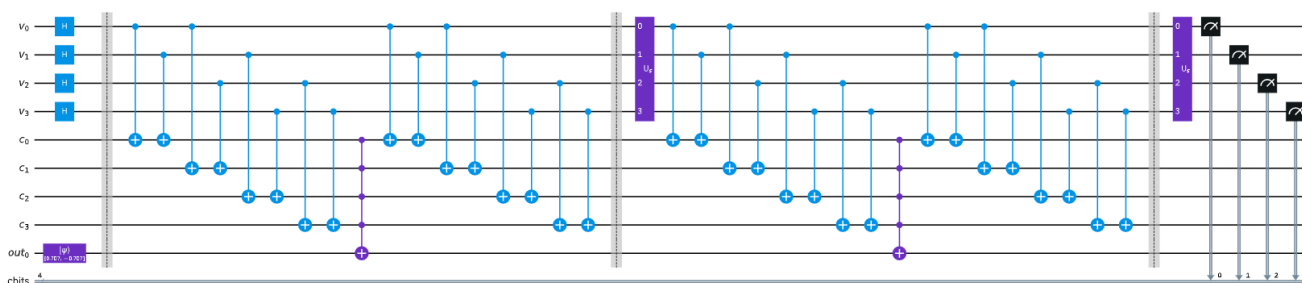
در این مرحله برای اینکه دوست داریم برای چک کردن اینکه جواب مسئله را پیدا کردیم یا نه تنها یک بیت داشته باشیم میتوانیم از یک multi controlled toffoli gate استفاده کنیم که در اینجا اگر تمام بیت های c_0 تا c_3 یک باشند خروجی out_0 برابر یک خواهد بود.

در نهایت در بخش پایانی نیز باید عمل uncompute را انجام دهیم یعنی باید مطمئن شویم که بیت های $c_0, c_1, c_2, c_3 = 0$



شکل ۷: استفاده از toffoli-gate در qiskit

شوند. این نکته نیز در اینجا شایان ذکر است که الگوریتم grover به شکل iterative به $radicaln$ اجرا برای یافتن جواب احتیاج دارد. در شکل آخر نیز میتوانید مدار کامل شده این مسئله را مشاهده کنید.

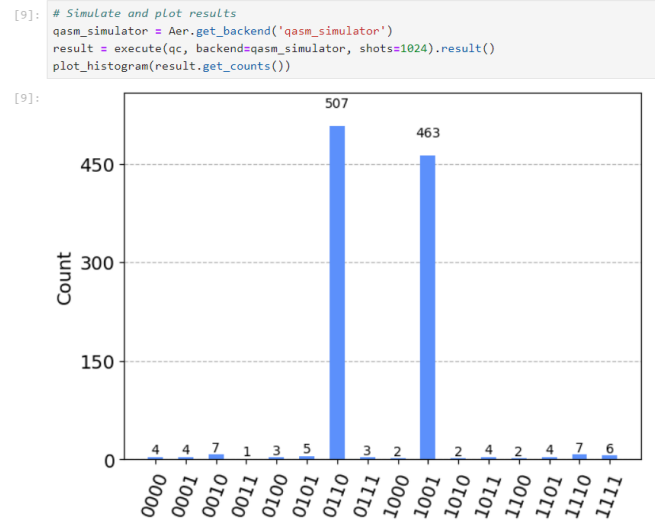


شکل ۸: مدار کامل این پروژه

در پایان نیز جهت تست کارکرد درست این مدار آن را به تعداد ۱۰۲۴ بار با یک simulator شبیه سازی کردیم و نتیجه آن در شکل زیر قابل رویت میباشد:

حال بیایید به تحلیل آن بپردازیم همان طور که در شکل مشخص است دو مقدار به تعداد بسیار بیشتری از بقیه موارد اندازه گیری شده اند که در واقع همان جواب های سودوکوی ۲ در ۲ ما میباشد.

There are two bit strings with a much higher probability of measurement than any of the others, 0110 and 1001. These correspond to the assignments: $v_0 = 0, v_1 = 1, v_2 = 1, v_3 = 0$ and $v_0 = 1, v_1 = 0, v_2 = 0, v_3 = 1$ which are the two solutions to our sudoku!



شکل ۹: خروجی نهایی

مراجع

- [1] “Quantum algorithm,” https://en.wikipedia.org/wiki/Quantum_algorithm, 2023.